



Advanced Database Systems:

Part III.2 Introduction to NoSQL Systems

Reinhard Pichler

Institute of Logic and Computation
"Databases & Artificial Intelligence" Group

Summer Semester 2023

References

❖ Textbooks:

- Pramod J. Sadalage and Martin Fowler: NoSQL Distilled. Addison-Wesley 2013
- Lena Wiese: Advanced Data Management for SQL, NoSQL, Cloud and Distributed Databases. de Gruyter, 2015

❖ Scientific Articles:

- Felix Gessert, Wolfram Wingerath, Steffen Friedrich, Norbert Ritter: NoSQL database systems: a survey and decision guidance. Computer Science - R&D 32(3-4): 353-365 (2017).

References (cont.)

❖ Further Scientific Articles:

- Eric A. Brewer: CAP twelve years later: How the "rules" have changed. IEEE Computer 45(2): 23-29 (2012)
- Seth Gilbert, Nancy A. Lynch: Brewer's conjecture and the feasibility of consistent, available, partition-tolerant web services. SIGACT News 33(2): 51-59 (2002)
- Daniel Abadi: Consistency Tradeoffs in Modern Distributed Database System Design: CAP is Only Part of the Story. IEEE Computer 45(2): 37-42 (2012)
- Dan Pritchett: BASE: An Acid Alternative. ACM Queue 6(3): 48-55 (2008)

❖ Copyright of all figures is with one of these resources.

Contents

- ❖ Motivation
- ❖ Four Categories of NoSQL Systems
- ❖ Further Aspects of Data Modelling
- ❖ Data Distribution and Consistency

Contents

❖ **Motivation**

- ❖ Four Categories of NoSQL Systems
- ❖ Further Aspects of Data Modelling
- ❖ Data Distribution and Consistency

Relational DBMSs

❖ Strengths:

- Strong formal foundation (logic, relational algebra)
- Standard data model and query language (SQL)
- Concurrency: support multi-user access; transactions
- Data integration: store data of many applications in one database

❖ Previous alternative proposals:

- XML Stores, Object-Oriented DBMSs
- Have been integrated into RDBMSs and complement/extend now their functionality

Limitations of Relational DBMSs

- ❖ **Schema**: must be known before any data can be added
- ❖ **Data structure**:
 - RDBMS are designed for structured data; limited support for unstructured data (text, pictures/videos)
 - Requires horizontal and vertical homogeneity of data (same structure per row, same type per column).
 - Data belonging to an entity is split into several tables
 - **Impedance mismatch**: relational model vs. in-memory data structures (e.g.: OO, nested structures)
- ❖ **Cost**:
 - major vendors: closed source with strict licensing fees
 - usual licensing model: pay per user / computing node

Time Consumption in RDBMSs

- ❖ M. Stonebraker et al. have found that in a typical RDBMS, only 6.8% of the execution time is spent on "useful work".
- ❖ The rest is spent on:
 - buffer management (34.6%): disk I/O
 - locking (16.3%): to control concurrency
 - latching (14.2%): to protect shared data structures from race conditions caused by multi-threading
 - logging (11.9%): to guarantee durability
 - other code parts (16.2%)
- ❖ Conclusion: the end of the "one-size-fits-all" era

Reference: Michael Stonebraker, et al.: The End of an Architectural Era (It's Time for a Complete Rewrite). VLDB 2007: 1150-1160

New Data Management Challenges

- ❖ **Volume**: processing "big data"
- ❖ **Complex data structures**:
 - Nested structures
 - Complex relationships (graph structure)
- ❖ **Schema Independence / Schema Evolution**:
 - to facilitate integration of data from various sources
 - coping with changing structure of the data
 - allow self-descriptiveness
- ❖ **Sparseness**:
 - Many data items are unknown or non-existent
 - Would result in many NULLs in relational tables

Reasons for Emergence of New DB Systems

❖ Application vs. integration database:

- Shift from idea of integration DB to application DB, i.e.: each application maintains its own database
- Data exchange via web services
- advantages of application DB: more flexibility (appropriate choice of data model and DB features)

❖ Use of Clusters

- Scale out to cope with huge amounts of data;
- RDBMSs are not well-suited for running on clusters.

The Term "NoSQL"

❖ History:

- June 2009: meetup in San Francisco for "open-source, distributed, non-relational databases".
 - organized at the time of the Hadoop Summit 2009;
 - search for a Twitter hashtag without many Google hits;
 - NoSQL was chosen
- ❖ Some of the Systems presented at this 2009 meetup:
<https://blog.oskarsson.nu/post/22996140866/nosql-debrief>
- Cassandra, HBase, Voldemort, CouchDB, MongoDB

The Term "NoSQL" (cont.)

- ❖ NoSQL = "Not only SQL"
- ❖ NoSQL as an umbrella term for all those DB systems that share **most of the following properties**:
 - open-source, distributed, non-relational
 - schemaless
 - restricted querying capabilities (e.g., no joins)
 - horizontally scalable; running well on clusters
 - no (or weaker form of) transactions, not guaranteeing ACID properties, use BASE instead

Pioneering Contributions by Companies

❖ Early systems:

- Google: Bigtable (column family store)
- Amazon: DynamoDB (key-value store)
- Facebook: Cassandra (column family store), later used by Twitter
- LinkedIn: Project Voldemort (open-source implementation of Amazon's DynamoDB)

❖ Characteristics of early systems:

- Huge number of users
- Huge number of reads/writes
- Non-critical transactional systems (ACID not needed)

Pioneering Contributions by Companies (cont.)

❖ Scientific Articles:

- Fay Chang, Jeffrey Dean, Sanjay Ghemawat, Wilson C. Hsieh, Deborah A. Wallach, Michael Burrows, Tushar Chandra, Andrew Fikes, Robert E. Gruber: [Bigtable: A Distributed Storage System for Structured Data](#). ACM Trans. Comput. Syst. 26(2): 4:1-4:26 (2008).
- Giuseppe DeCandia, Deniz Hastorun, Madan Jampani, Gunavardhan Kakulapati, Avinash Lakshman, Alex Pilchin, Swaminathan Sivasubramanian, Peter Vosshall, Werner Vogels: [Dynamo: Amazon's highly available key-value store](#). SOSP 2007: 205-220.
- Avinash Lakshman, Prashant Malik: [Cassandra: a decentralized structured storage system](#). Operating Systems Review 44(2): 35-40 (2010).

Benefits of NoSQL Database Systems

❖ High performance:

- Handling huge amounts of data
- Specific optimizations (e.g.: compression in column stores; path queries in graph databases)

❖ Horizontal scalability:

- Add nodes to the cluster as load or DB grows

❖ Schemaless:

- Simpler storage mechanisms: performance
- No schema needed upfront, flexible when data changes

❖ No (or weaker forms of) transactions:

- Simpler transaction handling; not blocking
- Reduced logging requirements

Disadvantages of NoSQL Systems

- ❖ No standardized Query Language:
 - Users need to learn a separate query language for each system (even within the same category)
- ❖ Weaker form (or no support of) transactions
 - no full ACID support (in particular, consistency)
- ❖ Limited support of ensuring integrity:
 - Responsibility for uniqueness constraints, referential integrity, etc. is shifted to the application.

Contents

- ❖ Motivation
- ❖ **Four Categories of NoSQL Systems**
- ❖ Further Aspects of Data Modelling
- ❖ Data Distribution and Consistency

Four Categories

❖ Key-value stores:

- Stores values indexed by keys

❖ Document stores:

- Similar to key-value stores but DBMS can understand the format of the value; typically JSON, XML

❖ Columnar databases, column family stores:

- ColumnarDB, column store: data stored in columns
- Column family store, wide column store: can be thought of as two-dimensional indexing of values: first by id (like key) then by column name

❖ Graph Databases:

- Uses graphs to represent data; vertices represent entities; edges represent (directed) relationships

Examples of Systems in the Four Categories

See <http://nosql-database.org/> (over 200 systems);
(see also <https://db-engines.com/en/ranking>)

❖ Key-value stores:

- Riak, Redis, BerkeleyDB, DynamoDB (Amazon); open source version: Project Voldemort

❖ Document stores:

- MongoDB, CouchDB

❖ Columnar databases, column family stores:

- MonetDB, HBase, Cassandra, Accumulo, Google Bigtable

❖ Graph Databases:

- Neo4J, TigerGraph

Complex Structures

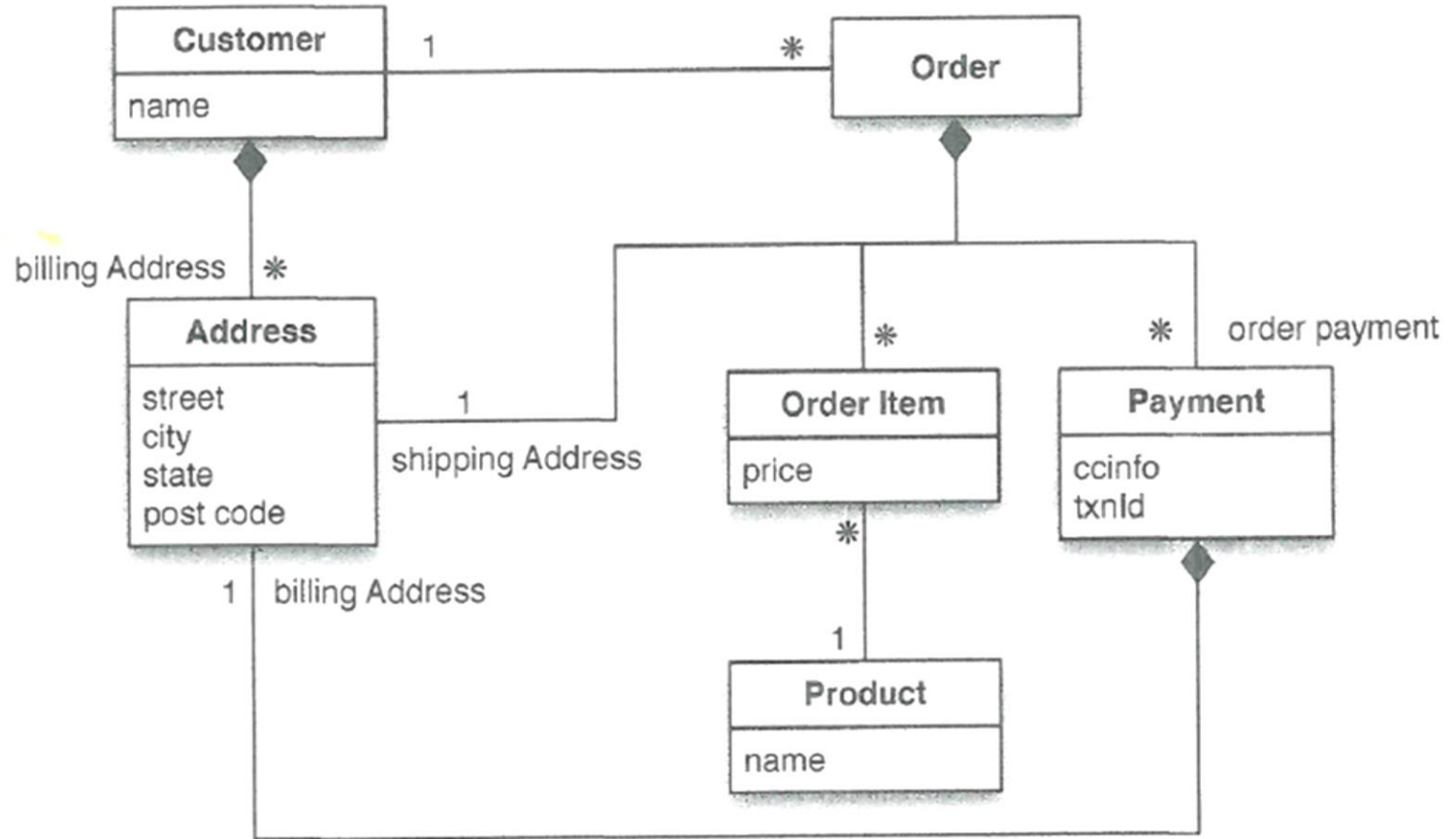
❖ "Aggregate Orientation":

- Data organization in RDBMS: by tuples
- many applications: access to more complex structures
- **aggregate**: = collection of objects that we want to treat as a unit, e.g. unit of access to storage, unit of updates, unit of communication
- in data modelling: **denormalization**
- main goal of NoSQL data modelling: read optimization

❖ Consequences of aggregate orientation:

- Well-suited for running on a cluster
- Consequence for transactions: easy to support atomic manipulation of single aggregate; difficult to support atomic manipulation of several aggregates

Complex Structures: Example



Complex Structures: Example (cont.)

```
// in customers
```

```
{  
  "id":1,  
  "name":"Martin",  
  "billingAddress":[{"city":"Chicago"}]  
}
```

```
// in orders
```

```
{  
  "id":99,  
  "customerId":1,  
  "orderItems": [  
    {  
      "productId":27,  
      "price": 32.45,  
      "productName": "NoSQL Distilled"  
    }  
  ],  
  "shippingAddress":[{"city":"Chicago"}]
```

```
// in customers
```

```
{  
  "customer": {  
    "id": 1,  
    "name": "Martin",  
    "billingAddress": [{"city": "Chicago"}],  
    "orders": [  
      {  
        "id":99,  
        "customerId":1,  
        "orderItems": [  
          {  
            "productId":27,  
            "price": 32.45,  
            "productName": "NoSQL Distilled"  
          }  
        ],  
        "shippingAddress":[{"city":"Chicago"}]
```

Aggregates in NoSQL Systems

❖ Key Value Stores:

- arbitrary structure of values: opaque to DB system; only known to the application.

❖ Document Stores:

- nested structure (XML, JSON) visible to the DB system

❖ Column-oriented DBs:

- distinguish column stores and wide-column stores
- usually, only the latter are considered as NoSQL

❖ Graph Databases:

- aggregate-ignorant (like RDBMSs)
- little information stored per entity; most important information lies in the relationships.

Contents

- ❖ Motivation
- ❖ Four Categories of NoSQL Systems
- ❖ **Further Aspects of Data Modelling**
- ❖ Data Distribution and Consistency

Relationships

❖ Data accesses:

- aggregates help to put together data that is commonly accessed together.
- Relationships between aggregates: usually, not all related data is combined in one aggregate; in particular, applications may have different aggregate boundaries.

❖ Example:

- Customers vs. orders: some application may access customer data together with order information; another application may access only the orders.
- Shall we store customers and orders as one aggregate or as separate aggregates?

Relationships (cont.)

❖ Modelling inter-aggregate relationships:

- IDs in one aggregate to reference other aggregates
- In general, this reference-semantics is not visible to the DB system; however, e.g. Riak allows link information in metadata and supports link-walking
- Graph DBs are optimized for traversing relationships.

❖ Challenge with inter-aggregate relationships:

- how to ensure atomicity of changes to related aggregates (ACID property of transactions in RDMBS)?

Materialized Views

❖ Materialized Views in RDBMSs:

- Lack of aggregate structure in RDBMS: easier to support different ways of accessing the data
- Views: virtual tables; convenient for encapsulation
- Materialized views: views cached on disk

❖ NoSQL Systems:

- Typically don't have views but some allow precomputed and cached queries (referred to as materialized views)

❖ Update strategies:

- Eagerly: whenever base data is updated
- Lazily: update materialized views at regular intervals

Schemaless Databases

❖ Freedom and flexibility in NoSQL Systems:

- key-value store: arbitrary contents in value
- document store: arbitrary structure of documents
- Column-family store: arbitrary combination of columns
- flexibility mainly applies to data within an aggregate, not when aggregate boundaries have to be modified.

❖ "Schemaless":

- schema on write (RDBMS) vs. schema on read
- implicit schema: DB system is ignorant of schema but application programs assume a schema
- may be difficult to maintain: requires understanding of the schema in the application code.

Contents

- ❖ Motivation
- ❖ Four Categories of NoSQL Systems
- ❖ Further Aspects of Data Modelling
- ❖ **Data Distribution and Consistency**

Main Aspects of Data Distribution

❖ General considerations:

- NoSQL Systems are mostly designed for running in a distributed environment;
- aggregates are the natural unit of distribution
- Goals: read and/or write performance, load balancing
- Challenges: consistency (and concurrency)

❖ Sharding:

- Distributing different data across multiple nodes
- $\approx$ horizontal fragmentation in DDBMS-terminology

❖ Replication:

- holding copies (replicas) of the data on multiple nodes
- primary-secondary vs. peer-to-peer replication.

Primary-Secondary Replication

❖ Characteristics:

- Writes processed by Primary; distributed to Secondaries
- Primary: bottleneck, single point of failure
- high read performance
- resilience against Secondary failure
- read resilience against Primary-failure: Secondaries can continue handling read requests.
- Special case hot-standby: all reads/writes handled by Primary; Secondary takes over on Primary failure

❖ Inconsistency problems:

- Different clients may read different values
- Possibly a client cannot read its own writes

Peer-To-Peer Replication

❖ Characteristics:

- all nodes accept writes and reads
- high read and write performance

❖ Additional inconsistency problems:

- Write-write conflicts in case of parallel writes of same data items on different servers
- Lost updates: update-2 overwrites update-1 although it was based on reading state before update-1.
- Coordination between replicas may cause high communication load.

Aspects of Consistency

❖ Write-write conflicts:

- **Pessimistic approach:** apply locking to prevent write-write conflicts; deadlock handling required
- **Optimistic approach:** allow write-write conflicts and merge updates later; domain-specific inconsistency handling required, e.g.: shopping carts in DynamoDB: user checks the cart before completing the order.

❖ Read-write conflicts:

- e.g.: unrepeatable read on single server
- replication consistency: same value on all replicas?

❖ Inconsistency window: time needed to restore consistency; sensitive to communication delays.

Strong Consistency

- ❖ Strong consistency on a single server:
 - Transactions with ACID properties
 - ensure serializability of transactions
- ❖ Strong consistency on a cluster:
 - DB system appears to the user as on a single server
- ❖ Compromises:
 - Even on a single server, strong consistency may be traded off for performance/throughput
 - Isolation levels in SQL: usually READ COMMITTED preferred over SERIALIZABLE
 - Tradeoff Consistency vs. Availability / Performance / Throughput is particularly important on a cluster.

Weaker Forms of Consistency

- ❖ **Eventual consistency** (see next slide)

- ❖ **Causal consistency:**

All nodes agree on order of potentially causally related operations. In particular, a read preceding a write is considered as a potential cause of the write.

- **Read-Your-Writes:** After writing data, user reads this (or later) data
- **Monotonic Reads:** After reading data, user will never read an older version from another replica
- **Monotonic Writes:** Subsequent writes of the same user are observed in this order by other users.
- **Writes Follow Reads:** User1 reads A and then writes B; If User2 reads B; then it also sees A (or later version).

ACID vs. BASE

❖ BASE property of transactions::

- **BA**: basically available: DB appears to work most of the time.
- **S**: soft state: system need not be in a consistent state all the time (write-write conflicts; differing replicas)
- **E**: eventual consistency: if no further updates arrive, system will eventually reach a consistent state.

❖ Intuitive meaning:

- The DB system works basically all the time;
- data does not have to be consistent all the time;
- but it will eventually reach a consistent state.
- Term "BASE" ($\neq$ "ACID") is mainly a play on words.

CAP Properties

❖ Consistency

- "Single-copy consistency": system behaves as if operations were processed one at a time on a single copy

❖ Availability

- Every request received by a non-failing node in the cluster must result in a response.

❖ Partition Tolerance

- System continues its operation in case of network partition.

CAP Theorem (due to Eric Brewer)

❖ Theorem:

In a cluster, you can have at most 2 of the 3 properties **C**onsistency, **A**vailability, and **P**artition Tolerance.

❖ Consequences

- Network partitions cannot be excluded: there is a C vs. A trade-off; this is not a binary decision but allows for different levels of C vs. A.
- Classify systems as AP (i.e., prefer availability over consistency in case of network partition) or CP.
- CA is impractical; it would mean total shut-down in case of a network partition.

Note on the History of the CAP Theorem

❖ Brewer (UC Berkeley):

- Late 1990's: presented as important principle
- PODC 2000: presented as a "Conjecture".

❖ Seth Gilbert and Nancy Lynch (MIT):

published a proof for both, asynchronous and partially synchronous networks (i.e., each node has a clock).

Proof idea:

- Assume network partitioned into G_1 , G_2 with no messages possible between them.
- Assume write on G_1 ; terminates by availability.
- Assume read on G_2 after termination of write in G_1 ; this also terminates and yields result different from G_1 .

PACELC Categorization

❖ Idea:

- CAP-Theorem only covers the trade-off A vs. C in case of network **partition**; it ignores the important problem of **latency** in a distributed system.
- Daniel Abadi: extends CAP to PACELC

❖ PACELC:

- **PAC**: A-C trade-off in case of network partition.
- **ELC**: **e**lse trade-off between **l**atency and **c**onsistency

❖ Categorization of (NoSQL) Systems:

- In principle, this gives rise to 4 types, e.g.: PA/EC: prefer A over C in case of partition and else C over L;
- possibly less realistic: PC/EL.

Quorum Consensus

- ❖ **Motivation:** define quorums to provide a flexible mechanism (balancing the performance of reading and writing) when ensuring consistency on a cluster:
 - **Read quorum R :** number of replicas that have to be contacted for a read operation to succeed;
 - **Write quorum W :** number of replicas that have to be contacted for a write operation to succeed.
- ❖ **Requirements** (assuming replication factor N):
 - **Overlap of two writes:** $W + W > N$, i.e., more than half of the replicas are reached by a write operation
 - **Overlap of write-read:** $R + W > N$, i.e., at least one node accessed by the read was involved in last write.

Quorum Consensus (cont.)

❖ Idea of the overlap:

- **Read request:** is guaranteed to see the most recent version of the data item (**version stamps required!**).
- **Write request:** since more than half of the nodes have to accept the write, concurrent writes are prevented.

❖ Extreme cases:

- **ROWA:** read-one, write-all: $W = N, R = 1$
- **Majority quorum:** $W \geq \lfloor N/2 \rfloor + 1$ and $R \geq \lfloor N/2 \rfloor$



Version Stamps

- ❖ **Challenge:** recognize in a distributed environment which version of a data item is older and which is newer.
- ❖ **Timestamps:**
 - simple but not reliable in distributed environment
 - synchronization of clocks on different nodes is complex
- ❖ **Logical Clocks:**
 - basically a counter for messages/events
 - clock tick = increment of counter
 - compare counter values to decide order of versions

Scalar Clocks (= Lamport Clocks)

❖ Based on "happened-before-relation":

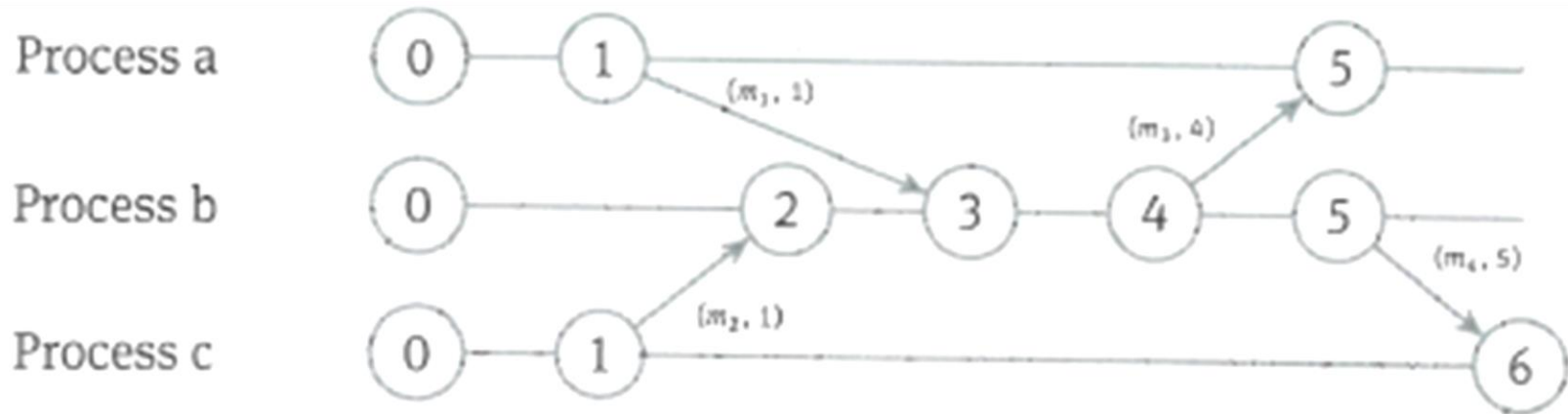
- induced by the total order of events on a single node (or, more fine-grained, of a single process)
- sending a message happens before receiving it
- take the transitive closure

❖ Notation: $e_1 \rightarrow e_2$ (i.e., event e_1 happened before event e_2)

❖ Scalar Clock (= Lamport Clock):

- Initialization: counter C_i on every server i is set to 0.
- Send: before server i sends a message, increment counter C_i and attach it to the message.
- Receive: when server j receives a message with counter C_i then set $C_j := \max(C_i, C_j) + 1$.

Scalar Clocks: Example



Remark:

- Global clock C of events: if event e occurs on node i , then we set $C(e) := C_i(e)$. We thus get a global order of all events in the cluster.
- Scalar clocks provide no total order;
- e.g.: in the above example, the messages sent by processes a and b have the same counter 1.

Weak / Strong Clock Property

❖ Weak clock property:

- For any two events e_1 and e_2 :
if $e_1 \rightarrow e_2$ then $C(e_1) < C(e_2)$.

❖ Strong clock property:

- For any two events e_1 and e_2 :
 $e_1 \rightarrow e_2$ if and only if $C(e_1) < C(e_2)$.

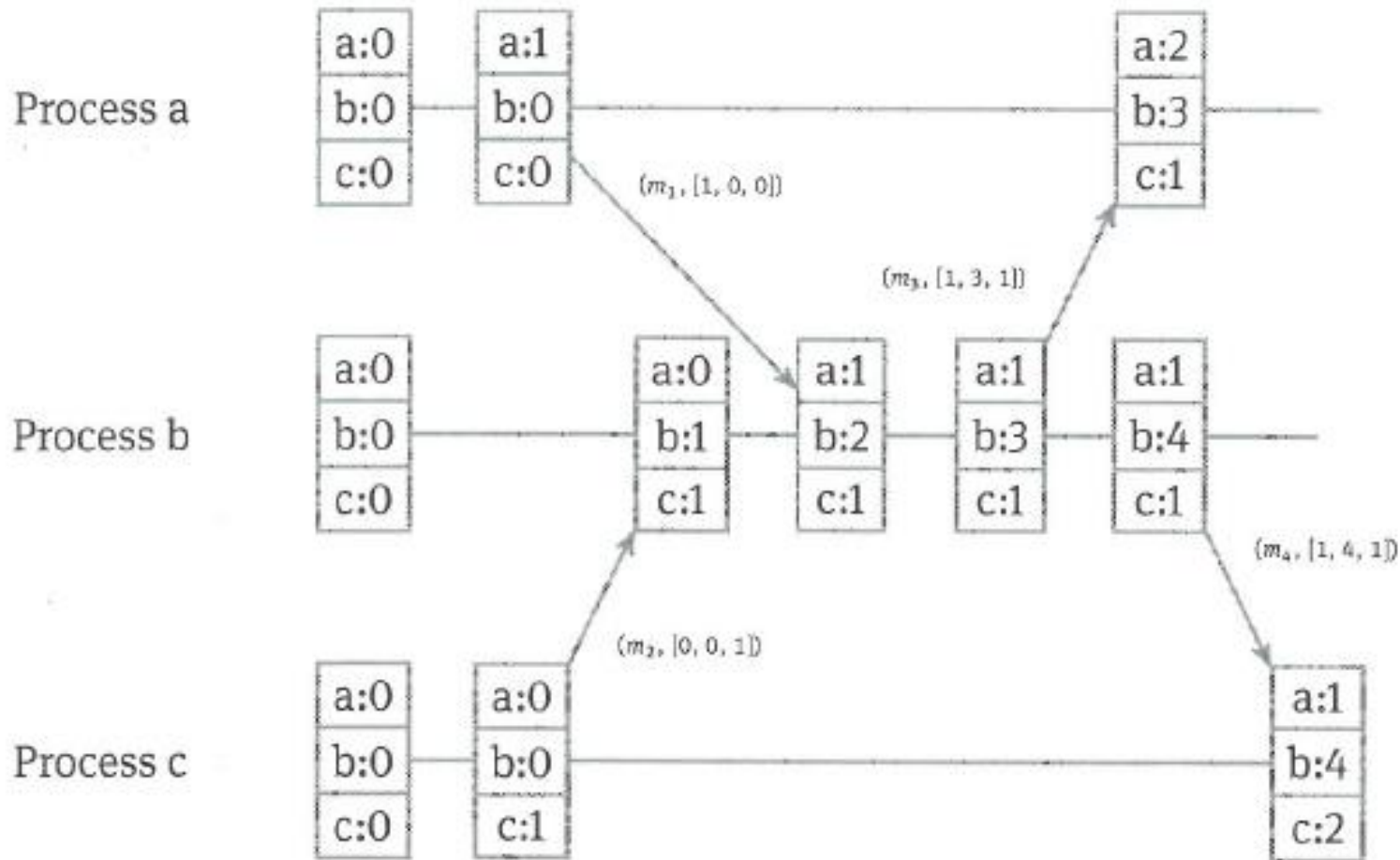
❖ Observation:

- Lamport clocks satisfy the weak clock property but not the strong clock property.
- This is due to independent sequences of events on different nodes.

Vector Clocks

- ❖ **Idea:** every node maintains a **vector of clocks** for all participating nodes.
- ❖ **Notation:** $VC_i[j]$ = clock of node j in vector at node i .
- ❖ **Maintaining a vector clock:**
 - Initialization: set $VC_i[j] := 0$ for every i and j .
 - Send: before server i sends a message, increment counter $VC_i[i]$ and attach VC_i to the message.
 - Receive: when server j receives a message from node i with vector VC_i , then do:
 - For every k , $VC_j[k] := \max(VC_i[k], VC_j[k])$.
 - Set $VC_j[j] := VC_j[j] + 1$.

Vector Clocks (Example)



Properties of Vector Clocks

❖ Order on Vector Clocks:

- $VC \leq VC'$ if and only if $VC[k] \leq VC'[k]$ for every k .
- $VC < VC'$ if and only if $VC \leq VC'$ and there exists at least one k , such that $VC[k] < VC'[k]$.

❖ Observation:

- Vector clocks satisfy the **strong clock property**.
- This is due to the fact that independent sequences of events on different nodes are assigned incomparable vectors;
- e.g. on the previous slide: the first messages sent by process a and by process c get incomparable vectors.

Summary

❖ Motivation:

- Relational DBMSs: strengths, limitations
- New challenges; reasons for new database systems
- integration vs. application database; end of "one size fits all"

❖ NoSQL systems:

- Characteristics
- Strengths and weaknesses of NoSQL systems
- 4 categories of NoSQL systems

❖ Data distribution: sharding, replication

Summary (cont.)

- ❖ Consistency:
 - strong consistency vs. weaker forms of consistency
 - ACID vs. BASE
 - CAP Theorem and its consequences
 - extension of CAP to PACELC
- ❖ Quorum consensus: motivation, read/write, requirements, special cases: ROWA, majority
- ❖ Version stamps: motivation, timestamps vs. logical clocks
- ❖ Scalar clocks (Lamport clocks) vs. vector clocks
- ❖ Weak/strong clock property